المدرسة الوطنية للعلوم التطبيقية - بني ملال

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏ ⵜⵎⵓⵙⵙⵏⴰⵡⵉⵏ ⵜⵉⵏⵎⵍⴰⵍⵉⵏ - ⴰⵎⵜ ⵎⵉⵍⴰⵍ

Ecole Nationale des Sciences Appliquées - Béni Mellal

COMPTE RENDU DE TRAVAUX PRATIQUE PYTHON

1ère Année – Filière Intelligence Artificielle et Cybersécurité
Année Universitaire 2025–2026

Travail réalisé par : Achraf Hamouda

Sous l'encadrement de : Mme. Ibtissame Aouraghe

1 NOVEMBRE 2025

ÉCOLE NATIONALE DES SCIENCES APPLIQUEES DE BENI MELLAL
Université Sultan Moulay Slimane

Table des matières

1ère Année – Filière Intelligence Artificielle et Cybersécurité Année Universitaire 2025/2026 0

Introduction 2

Exercice 1 : Nombres Parfaits 3

Exécution1 : 4

Remarque 4

Exercice 2 : Équation du second degré 5

Exécution2 : 6

Exercice 3 : Somme des chiffres jusqu'au palindrome 7

Exécution3 : 9

Exercice 4 : Suite de Collatz 10

Exécution4 : 11

Exercice 5 : Division entière 12

Exécution5 : 13

Exercice 6 : Nombre de Kaprekar 14

Exécution6 : 16

Exercice 7 : Nombre Amicaux 17

Exécution7 : 18

Exercice 8 : Suite de Fibonacci 19

Exécution8 : 20

Introduction

Ce TP s'inscrit dans le cadre du module de programmation Python, dont l'objectif est de nous initier à la logique algorithmique à travers un langage moderne, lisible et puissant, largement utilisé dans les domaines de l'intelligence artificielle, de la cybersécurité et de la science des données.

Après le premier TP consacré à l'installation de l'environnement de travail et à la découverte des interfaces Spyder et Jupyter Notebook, nous avons exécuté nos premières commandes dans un langage fascinant : Python, conçu par Guido van Rossum.

Dans ce troisième TP, nous approfondissons notre apprentissage en Python en nous concentrant sur la manipulation des boucles, des suites numériques et de la logique conditionnelle avancée. L'objectif est de renforcer notre capacité à traduire un raisonnement algorithmique en code exécutable.

Pour la réalisation de ce TP, nous avons utilisé Visual Studio Code (VS Code), un environnement de développement moderne et polyvalent qui offre plusieurs avantages.

Nombres Parfaits

Exercice 1 : Nombres Parfaits

Cet exercice consiste à rechercher les cinq premiers nombres parfaits en calculant la somme de leurs diviseurs propres.

exo1.py > nombre

```python
def nbr_parfait(x) :
    somme = 0
    for i in range(1,x) :
        if x % i == 0 :
            somme += i
    return somme == x
nombre = 1
compt = 0
while compt < 5 :
    if nbr_parfait(nombre) :
        print(nombre)
        compt +=1

    nombre += 1
```

Figure 1: code source

- on définit une fonction nbr_parfait(x) qui calcule la somme des diviseurs propres d'un nombre x. Pour cela, on utilise une boucle for allant de 1 jusqu'à x-1 et on ajoute tous les diviseurs dans une variable somme.
- Ensuite, la fonction retourne true si la somme des diviseurs est égale à x, ce qui signifie que le nombre est parfait, sinon false.
- Puis, dans le programme principal, on initialise deux variables : nombre, qui représente l'entier en cours de test, et compt, qui compte combien de nombres parfaits ont été trouvés.

- Puis, On utilise une boucle while compt < 5 afin de continuer le processus jusqu'à afficher cinq nombres parfaits. À chaque itération, on vérifie avec la fonction si nombre est parfait.
- Si c'est le cas, on l'affiche et on incrémente le compteur compt pour signaler qu'un nombre parfait supplémentaire a été trouvé.
- À la fin de chaque passage dans la boucle, on incrémente nombre pour tester le nombre suivant.
- La boucle s'arrête automatiquement après la détection des 5 premiers nombres parfaits.

Exécution1 :

```
n.exe c:/Users/achra/Desktop/tp3_py/exo1.py
6
28
496
8128
```

Figure 2

Remarque : Le 5eme nombre parfait est énorme, donc le programme met beaucoup de temps à le trouver.

Équation du second degré

Exercice 2 : Équation du second degré

Nous résolvons une équation du second degré en utilisant le discriminant afin de déterminer le nombre et la nature de ses racines.

```
exo2.py > ...
import math

a = float(input("Entrer la valeur de a : "))
b = float(input("Entrer la valeur de b : "))
c = float(input("Entrer la valeur de c : "))

if a == 0:
    print("Ce n'est pas une équation du 2e degré.")
else:
    delta = b**2 - 4*a*c

    if delta > 0:
        x1 = (-b - math.sqrt(delta)) / (2*a)
        x2 = (-b + math.sqrt(delta)) / (2*a)
        print(f"Deux solutions réelles distinctes : x1 = {x1} et x2 = {x2}")
    elif delta == 0:
        x = -b / (2*a)
        print(f"Une seule solution réelle double : x = {x}")
    else:
        print("Pas de solution réelle (Δ < 0).")
```

Figure 3 : code source

- On importe le module math afin d'utiliser la fonction sqrt().
- Ensuite, le programme demande à l'utilisateur de saisir les valeurs réelles a, b et c via des input(), puis les convertit en nombres flottants pour gérer les calculs avec précision.
- Puis, on vérifie si a == 0. Dans ce cas, l'équation n'est plus du second degré (elle devient du premier degré), donc un message d'erreur est affiché et le programme s'arrête.
- Si a est différent de zéro, on calcule le discriminant. Ce discriminant sert à déterminer le nombre et la nature des solutions.

- if delta > 0 permet de tester si le discriminant est strictement positif : dans ce cas, l'équation possède deux solutions réelles distinctes, calculées avec la formule du discriminant. Le programme affiche ensuite x1 et x2.
- Si delta == 0, alors l'équation possède une seule solution réelle double. Le programme calcule et affiche cette unique racine.
- Enfin, si delta < 0, l'équation n'admet aucune solution réelle, car la racine carrée d'un nombre négatif est impossible dans R.

Exécution2 :

```
n.exe c:/Users/achra/Desktop/tp3_py/exo2.py
Entrer la valeur de a : 1
Entrer la valeur de b : 0
Entrer la valeur de c : 4
Pas de solution réelle (Δ < 0).
PS C:\Users\achra\Desktop\tp3_py> & C:/Users/achra/AppData/Local/Programs/Python/Python313/pytho
n.exe c:/Users/achra/Desktop/tp3_py/exo2.py
Entrer la valeur de a : 1
Entrer la valeur de b : 0
Entrer la valeur de c : -4
Deux solutions réelles distinctes : x1 = -2.0 et x2 = 2.0
PS C:\Users\achra\Desktop\tp3_py>
```

Figure 4 : Cas de réussite et d'échec.

Somme des chiffres jusqu'au palindrome

Exercice 3 : Somme des chiffres jusqu'au palindrome

Cet exercice transforme un nombre en répétant l'ajout de la somme de ses chiffres jusqu'à obtenir un palindrome, tout en comptant les itérations.

```
exo3.py > ...
def somme_chiffres(x):
    somme = 0
    for a in str(x):
        somme += int(a)
    return somme

def palindrome(n):
    inverse = 0
    temp = n
    while temp > 0:
        d = temp % 10
        inverse = inverse * 10 + d
        temp //= 10
    return inverse

n = int(input("Entrer votre nombre : "))
iteration = 0

while True:
    if n == palindrome(n):
        print("Le nombre palindrome obtenu est :", n)
        print("Nombre d'itérations nécessaires :", iteration)
        break

    else:
        n += somme_chiffres(n)
        iteration += 1
        print(f"Étape {iteration} → Nouveau nombre = {n}")
```

Figure 5: code source

- On définit une fonction somme_chiffres(x) qui transforme le nombre en chaîne de caractères afin de parcourir chaque chiffre, puis additionne ces chiffres un par un pour retourner leur somme.
- Ensuite, on crée une seconde fonction palindrome(n) qui construit l'inverse du nombre. Pour cela, on utilise une boucle while : à chaque itération, on extrait le dernier chiffre avec % 10, on le rajoute à la fin du nombre inversé, puis on divise temp par 10 pour enlever ce chiffre.
- Le programme principal commence en demandant à l'utilisateur de saisir un entier positif et initialise une variable iteration permettant de compter le nombre d'étapes effectuées.
- Puis, on utilise une boucle while True afin de répéter indéfiniment les opérations jusqu'à l'obtention d'un palindrome.
- À chaque tour, on compare la valeur de n avec sa version inversée. Si elles sont égales, cela signifie que n est un palindrome : on l'affiche ainsi que le nombre total d'itérations, puis break interrompt la boucle.
- Dans le cas contraire (else), on applique la règle de l'énoncé : on calcule la somme des chiffres de n, on l'ajoute à n, puis on incrémente le compteur iteration. Un message d'information est affiché pour suivre l'évolution du nombre.
- Le processus continue jusqu'à ce qu'un palindrome soit obtenu, ce qui illustre l'utilisation des boucles, des conditions et de la manipulation des chiffres d'un nombre.

Exécution3 :

```
Entrer votre nombre : 726
Étape 1 → Nouveau nombre = 741
Étape 2 → Nouveau nombre = 753
Étape 3 → Nouveau nombre = 768
Étape 4 → Nouveau nombre = 789
Étape 5 → Nouveau nombre = 813
Étape 6 → Nouveau nombre = 825
Étape 7 → Nouveau nombre = 840
Étape 8 → Nouveau nombre = 852
Étape 9 → Nouveau nombre = 867
Étape 10 → Nouveau nombre = 888
Le nombre palindrome obtenu est : 888
Nombre d'itérations nécessaires : 10
```

Figure 6

Suite de Collatz

Exercice 4 : Suite de Collatz

Ce programme génère la suite de Collatz à partir d'un entier positif et compte le nombre d'étapes nécessaires pour atteindre 1.

```
exo4.py > ...
n = int(input("Entrer votre nombre : "))
iteration = 0
while n != 1 :
    iteration +=1
    if n % 2 == 0 :
        n = n // 2
        print("Iterartion : ",iteration," ; n = ", n)
    else :
        n = (n * 3) + 1
        print("Iterartion : ",iteration," ; n = ", n)
```

Figure 7: code source

- Le programme demande à l'utilisateur de saisir un nombre entier positif n Ensuite, une variable iteration est initialisée à 0 afin de compter le nombre d'étapes effectuées.
- Le programme utilise ensuite une boucle while pour répéter les opérations tant que la valeur de n n'est pas égale à 1, comme demandé dans la définition de la suite de Collatz. À chaque passage dans cette boucle, le compteur iteration est incrémenté pour indiquer qu'une nouvelle transformation a été effectuée.
- Une condition if permet de tester si n est pair. Si n % 2 == 0, cela signifie que n est divisible par deux : il est alors remplacé par n // 2.
  Le programme affiche ensuite la valeur obtenue ainsi que le numéro de l'itération.

- Sinon (else), cela signifie que n est impair. La règle de Collatz s'applique alors en remplaçant n par 3*n + 1. Le programme affiche également la nouvelle valeur suivie de son numéro d'itération.
- La boucle se répète ainsi jusqu'à atteindre n = 1, moment auquel le programme s'arrête automatiquement.

Exécution4 :

```
Entrer votre nombre : 21
Iterartion :  1  ; n =  64
Iterartion :  2  ; n =  32
Iterartion :  3  ; n =  16
Iterartion :  4  ; n =  8
Iterartion :  5  ; n =  4
Iterartion :  6  ; n =  2
Iterartion :  7  ; n =  1
PS C:\Users\achra\Desktop\tp3_py>
```

Figure 8

Division entière

Exercice 5 : Division entière

Nous effectuons une division entière à l'aide de soustractions répétées, sans utiliser les opérateurs / ni %.

```
exo5.py > ...
a = int(input("Entrer un nombre a positif : "))
b = int(input("Entrer un nombre b positif : "))

if b == 0 : print("Impossible")
elif a == 0 : print("La division entiere = 0")
elif b > a : print("La division entiere = 0")

z = 1
while b * z <= a :
    quotient = z
    z += 1
print("La division entiere de",a," par ",b, "est ",quotient)
```

Figure 9: code source

- Le programme demande deux entiers positifs a et b puis vérifie quelques cas particuliers : si b == 0, la division est impossible, et si a == 0 ou b > a, le quotient vaut 0. Dans le cas général, une variable z est utilisée pour compter combien de fois b peut être multiplié sans dépasser a. Une boucle while incrémente z tant que b * z <= a, et la dernière valeur valide de z correspond à la division entière de a par b.

Exécution5 :

```
Entrer un nombre a positif : 42
Entrer un nombre b positif : 7
La division entiere de 42  par  7 est  6
PS C:\Users\achra\Desktop\tp3_py>
```

Figure 10

Nombre de Kaprekar

Exercice 6 : Nombre de Kaprekar

Dans cette exercice , nous vérifions si un nombre satisfait la propriété de Kaprekar et affichons les dix premiers nombres correspondants.

```
exo6.py > ...
def kaprekar(x):
    if x == 1:
        print("Votre nombre est un Kaprekar")
        return

    y = x * x
    l = len(str(x))


    droite = y % (10 ** l)
    gauche = y // (10 ** l)

    if gauche + droite == x:
        print("Votre nombre est un Kaprekar")
    else:
        print("Votre nombre n'est pas un Kaprekar")

a = int(input("Entrer votre valeur : "))
kaprekar(a)
print("Les 10 premiers nombres de Kaprekar sont :")
count = 0
n = 1

while count < 10:
    y = n * n
    l = len(str(n))
    droite = y % (10 ** l)
    gauche = y // (10 ** l)

    if gauche + droite == n:
        print(n)
        count += 1
    n += 1
```

- Le programme définit une fonction kaprekar(x) permettant de vérifier si un nombre possède la propriété de Kaprekar. Si la valeur saisie est 1, le programme l'accepte directement, car 1 est un cas particulier connu. Dans le cas général, le programme calcule le carré du nombre puis récupère le nombre de chiffres de la valeur d'origine afin de déterminer où séparer ce carré.
- Ensuite, le carré est découpé en deux parties : la partie droite est obtenue à l'aide d'un modulo, tandis que la partie gauche est obtenue par une division entière. Si la somme de ces deux parties est égale au nombre initial, alors celui-ci est identifié comme un nombre de Kaprekar ; sinon, un message indique le contraire.
- Après la saisie de la valeur par l'utilisateur et l'appel de la fonction, une seconde partie du programme permet d'afficher les dix premiers nombres de Kaprekar. Pour cela, une boucle while incrémente progressivement un compteur n, applique la même méthode de séparation et teste la propriété à chaque itération. À chaque fois qu'un nombre valide est trouvé, il est affiché et le compteur principal est augmenté. La boucle s'arrête automatiquement après la découverte des dix premiers nombres de Kaprekar.

Exécution6 :

```
Entrer votre valeur : 31
Votre nombre n'est pas un Kaprekar
Les 10 premiers nombres de Kaprekar sont :
1
9
45
55
99
297
703
999
2223
2728
PS C:\Users\achra\Desktop\tp3_py>
```

Figure 11

Nombre Amicaux

Exercice 7 : Nombre Amicaux

Cet exercice recherche les trois premières paires de nombres amicaux en comparant la somme de leurs diviseurs propres.

```
exo7.py > ...
def amicaux():

    def somme_diviseurs(n):
        s = 0
        k = 1
        while k < n:
            if n % k == 0:
                s += k
            k += 1
        return s

    a = 2
    couples_trouve = 0
    print("Les trois premiers couples de nombres amicaux sont :\n")

    while couples_trouve < 3:
        b = somme_diviseurs(a)
        if b > a and somme_diviseurs(b) == a:
            couples_trouve += 1
            print(f" - Couple {couples_trouve} : ({a}, {b})")
        a += 1

amicaux()
```

Figure 12: code source

- Dans cet exercice, le programme commence par définir une fonction interne somme_diviseurs(n) qui calcule la somme des diviseurs propres d'un nombre n à l'aide d'une boucle while.

- Ensuite, deux variables sont initialisées : a, qui représente le nombre à analyser, et couples_trouve, qui servira à compter combien de couples de nombres amicaux ont déjà été identifiés. Une boucle while couples_trouve < 3 est ensuite utilisée pour parcourir progressivement les entiers à partir de 2.
- À chaque itération, le programme calcule la somme des diviseurs de a et la stocke dans b. Si b est supérieur à a (pour éviter les doublons) et si la somme des diviseurs de b est égale à a, alors les deux nombres forment un couple amical. Dans ce cas, le couple est affiché et le compteur est incrémenté.
- Enfin, la variable a est augmentée de 1 pour analyser le nombre suivant, et le processus continue jusqu'à ce que les trois premiers couples de nombres amicaux soient trouvés.

Exécution7 :

```
Les trois premiers couples de nombres amicaux sont :

 - Couple 1 : (220, 284)
 - Couple 2 : (1184, 1210)
 - Couple 3 : (2620, 2924)
PS C:\Users\achra\Desktop\tp3_py>
```

Figure 13

Suite de Fibonacci

Exercice 8 : Suite de Fibonacci

Nous générons une suite inspirée de Fibonacci à partir de deux valeurs initiales et affichons son neme terme .

```
exo8.py > ...
a = int(input("Entrer un nombre a : "))
b = int(input("Entrer un nombre b : "))
n = int(input("Entrer votre nombre n : "))

if n == 1:
    print(a)
elif n == 2:
    print(b)

else :
    f1 = a
    f2 = b
    c = 3
    while c <= n :
        fc = f1*f1 + f2*f2
        f2 = f1
        f1 = fc
        c += 1
    print(fc)
```

Figure 14: code source

- Le programme demandee à l'utilisateur de saisir deux valeurs initiales a et b, ainsi qu'un entier n correspondant au rang recherché dans la suite. Si n vaut 1 ou 2, le programme

affiche directement la valeur correspondante, car ces deux premiers termes sont déjà définis.

- Dans le cas contraire, deux variables f1 et f2 sont initialisées avec les valeurs saisies. Une boucle while est ensuite utilisée pour calculer chaque nouveau terme de la suite jusqu'au rang n. À chaque itération, le terme courant est obtenu en additionnant le carré des deux termes précédents (fc = f1*f1 + f2*f2), puis les valeurs sont décalées pour préparer le calcul suivant.
- Lorsque le compteur atteint la valeur n, la boucle s'arrête et le programme affiche le terme correspondant de la suite. Ce code illustre l'utilisation de variables intermédiaires et d'une boucle pour générer une suite numérique modifiée.

Exécution8 :

```
Entrer un nombre a : 4
Entrer un nombre b : 9
Entrer votre nombre n : 5
88840034
PS C:\Users\achra\Desktop\tp3_py>
```

Figure 15

FIN